



# Whisper-Lite

*TLS-secured chat for Android, built with Qt 6.10*

---

Sumith Shaha

Qt Advanced · TT00GD57-3001 · Metropolia University of Applied Sciences · May 2026 · Student

# Scope decision, owned upfront

Project plan v2.1 proposed:

**Full end-to-end encryption with libsodium**



Delivered:

**TLS-only, with E2EE deferred**

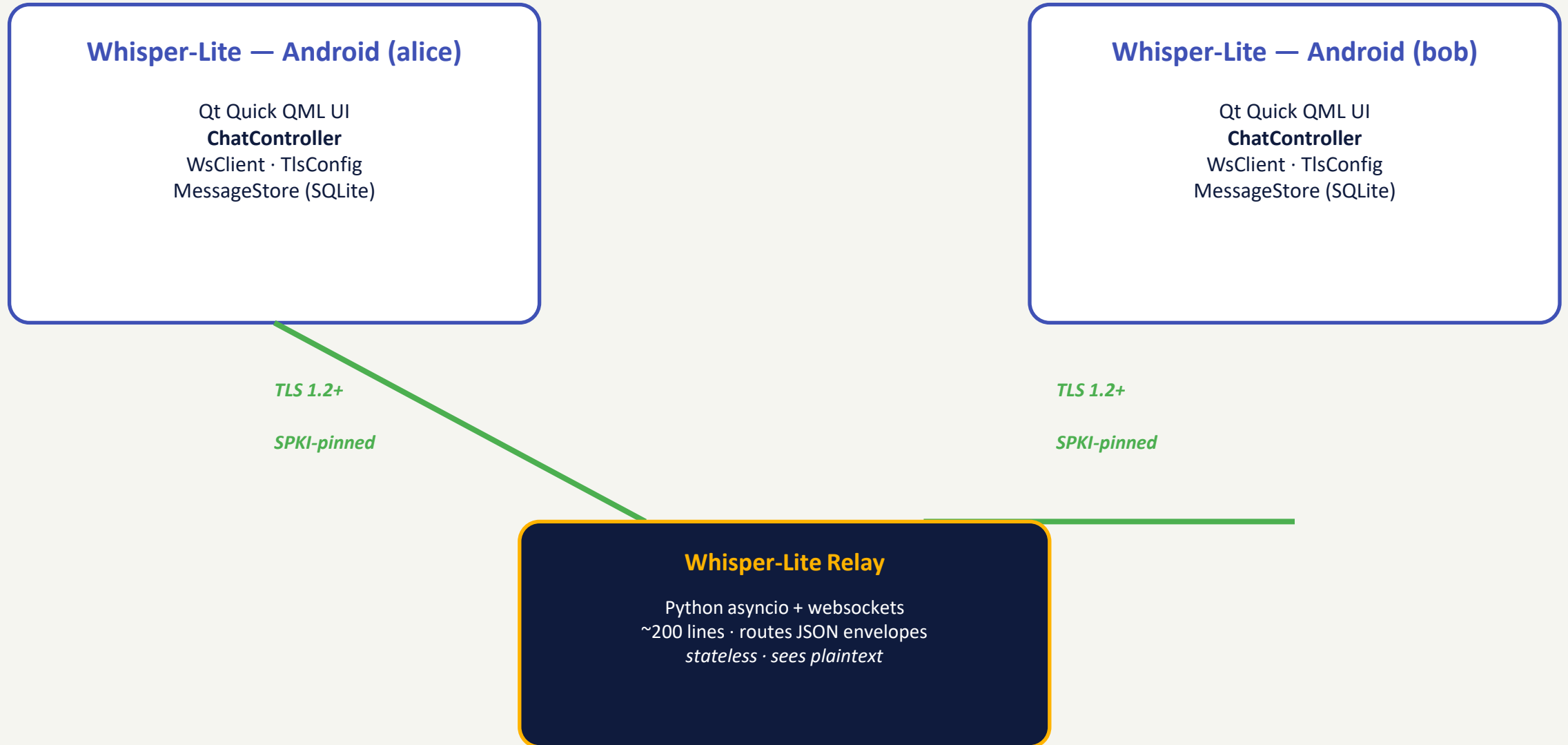
## Why I made this choice

The libsodium Android cross-compile + SPKI pin format mismatch between Android NSC and Qt cost me ~18 hours and I still didn't have a clean integration. With three weeks left, I had two paths:

- Ship E2EE half-working — visible bugs, demo failures likely
- Ship TLS-only end-to-end — every feature works, room for tests

*I picked option two. I'd rather hand Tero a smaller thing that works than a bigger thing that doesn't.*

# Architecture



*Server is intentionally minimal — every line of server code is a line that could leak metadata*

# TLS pinning — defence in depth

1

## Android Network Security Config

Bundled CA cert at `res/raw/whisper_ca.pem` registered as a debug-only trust anchor.

*Release builds ignore this entirely — a permissive config cannot accidentally ship to production.*

2

## App-level SPKI pin

`TlsConfig::acceptablePinned()` computes SHA-256(`SubjectPublicKeyInfo`) of the peer cert and compares against a baked-in pin in `res/raw/spki_pin.txt`.

*Even a malicious CA signing the same hostname would still fail the pin.*

***Either layer alone is defeatable. Together: an attacker has to modify the APK itself.***

# The Qt / QML boundary

*All QML interaction goes through ONE Q\_OBJECT — ChatController.*

```
class ChatController : public QObject {  
    Q_PROPERTY(MessageModel* messages READ messages)  
    Q_PROPERTY(UserModel* users READ users)  
    Q_PROPERTY(QString status NOTIFY statusChanged)  
    Q_INVOKABLE void sendText(QString peer, QString text);  
    Q_INVOKABLE void selectPeer(QString peer);  
};
```

1

## Testable

Unit-test C++ with no QML engine.

2

## One audit point

Every failure path lives here.

3

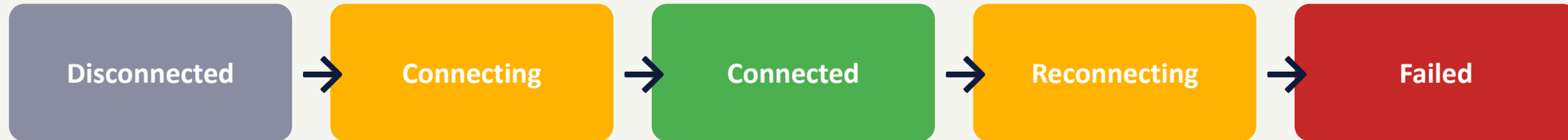
## Future-proof

E2EE re-enters in two functions.

### WHISPER\_QML\_BUILD compile guard

*Same headers compile cleanly in unit tests (no QML registration noise) AND in the full app (with QML\_ELEMENT). One source tree, two compile contexts — no duplication.*

# Networking — closed state machine, jittered backoff



## Exponential backoff with $\pm 20\%$ jitter

500ms · 1s · 2s · 4s · 8s · 16s · 30s (capped) · ... · 12 attempts max

### Why jitter?

*Without jitter, a flaky network becomes a thundering herd: 100 clients all retry at exactly 1s, then 2s, then 4s.  $\pm 20\%$  spreads them.*

File: `src/wsclient.cpp::scheduleReconnect`

### Heartbeat + TLS error policy

`QWebSocket::ping` every 25s while Connected

*Each TLS error has an explicit decision:*

- `HostNameMismatch` — debug-build only
- `SelfSigned` — only after pin matches
- **everything else — REJECT**

→ Demoing this live: kill server, watch amber, restart, watch green ←

# Tests — small but real

```
$ ctest --output-on-failure
```

```
Test #1: tst_store ..... Passed (7/7)
```

- ✓ **openCreatesSchema** Fresh DB gets the right tables and indexes
- ✓ **insertAndLoad** Round-trip insert → query → equal
- ✓ **orderingIsByTimestamp** Explicit ORDER BY ts (not insert order)
- ✓ **sqlInjectionDefeated** Bound parameters defeat injection — peer = '; DROP TABLE messages; --
- ✓ **knownPeers** DISTINCT peers, alphabetical

*Most important test: `sqlInjectionDefeated`. Stores `'; DROP TABLE messages; --` as a peer name. Verifies the table survives. Bound parameters make it trivially safe — but if someone refactors later, the test catches it.*

# Where E2EE plugs in — the architectural seam

*Two functions in ChatController.cpp. Roughly 20 lines.*

```
// In sendText(), instead of putting plaintext in the envelope:
const QByteArray blob = m_crypto->encryptFor(peer, m_myNick, text.toUtf8());
JsonObject env{
    { "type", "cipher" }, { "to", peer },
    { "blob", QString::fromLatin1(blob) }, // base64, opaque
    { "seq", ++m_seq }, { "ts", ts },
};

// In handleCipher(): just decrypt, then carry on
const QByteArray plain = m_crypto->decryptFrom(from, m_myNick, blob);
if (plain.isEmpty()) return; // MAC failure – drop silently
```

**Crypto choice:** libsodium crypto\_kx (X25519) for per-pair session keys + crypto\_secretbox\_easy (XSalsa20-Poly1305) per message. *Lex-smaller nick = libsodium client role, deterministic without a handshake.*

# Some Future Plans

1

## Add E2EE back via the seam in slide 7

*Plus persistent identity keys in Android Keystore*

2

## Group chat

*Signal sender keys for the simple version, MLS (RFC 9420) for the proper one*

3

## Real production deployment

*Drop the self-signed cert. Let's Encrypt on a real domain on CSC cPouta*

4

## WebSocket reconnect tests

*Bite the bullet on time-mocking. The reconnect logic deserves coverage*



# Thank you for your time.

---

Sumith Shaha · [sumithshaha.shaha@metropolia.fi](mailto:sumithshaha.shaha@metropolia.fi), [sumithshaha@gmail.com](mailto:sumithshaha@gmail.com)